Recursive Substrate Co-Design

A Research Proposal for Successive Machine-Designed Computational Substrates

David Holbert

Independent Researcher

Version 0.1 | September 30, 2026

Concept Paper / Research Proposal

This paper proposes a research framework and experimental program. It does not report completed RSCD experiments or claim empirical validation.

Abstract

Modern artificial intelligence is built on computational systems that were not originally designed for artificial intelligence. Today's AI inherits a long chain of historical engineering decisions: digital logic, separated memory and processing, fixed instruction sets, programming languages designed for humans, operating systems, numerical libraries, accelerators optimized for matrix operations, and machine-learning architectures shaped partly by the hardware available to execute them. Recent research has begun to relax these assumptions. Artificial intelligence can now assist in hardware design, discover learning algorithms, participate in hardware-software co-design, and operate on unconventional computing substrates such as neuromorphic and optical systems. However, these lines of work are usually studied separately. This proposal introduces Recursive Substrate Co-Design (RSCD): a research framework in which one generation of artificial intelligence jointly searches for a computational substrate, learning architecture, communication structure, and learning rule for a successor intelligence. The successor is then trained natively within that discovered substrate rather than merely ported onto it, and the process may repeat. The central hypothesis is that a successor intelligence trained natively on a machine-designed computational substrate may gain access to design spaces that were difficult or inaccessible to its predecessor. The first stage, Generation Zero, can be investigated entirely through simulation using current hardware.

Keywords: recursive self-improvement; hardware-software co-design; unconventional computing; neuromorphic computing; algorithm discovery; evolutionary search; computational substrates

1. Motivation

The conventional computing stack is extraordinarily capable, but it is also historically contingent. A simplified lineage is: physical switches -> digital logic -> processors -> memory hierarchy -> instruction sets -> operating systems -> programming languages -> numerical libraries -> accelerators -> neural networks -> foundation models.

Each layer influences the design of the layers above it. Modern neural architectures therefore do not exist independently of hardware. Matrix multiplication is central to contemporary AI partly because digital accelerators execute it extremely efficiently. Memory limits influence context lengths and batch sizes. Numerical formats influence training behavior. Parallel hardware favors certain network structures over others.

This is not evidence that contemporary computing is poorly designed. It is evidence that our current systems are products of a particular technological lineage. There is no strong reason to assume that this lineage represents the only effective way to organize computation for machine intelligence.

> ***The research question is therefore not simply how existing computers can run AI more efficiently. It is: What computational structures would emerge if an automated designer were allowed to search more deeply across the assumptions underlying computation itself?***

2. Existing Research Already Points in This Direction

2.1 Recursive AI improvement

Recent systems increasingly treat the development process itself as something AI can improve. Frontis-MA1 presents an AI-for-AI system that combines learned operators and evolutionary search to improve machine-learning engineering processes, explicitly framing machine-learning engineering as a testbed for recursive self-improvement [1]. This establishes that an AI system can participate recursively in improving the processes used to build later AI systems. However, the work remains primarily within software and machine-learning engineering on conventional computational infrastructure. RSCD extends the recursion downward into the computational substrate.

2.2 AI-driven hardware/software co-design

Hardware design is also becoming an increasingly automated search problem. The 2026 CHIA framework integrates AI agents, simulators, system-on-chip design tools, evolutionary coding systems, and conventional hardware-development workflows into cyclic design processes. Its case studies include evolutionary architectural discovery and AI-directed microarchitectural optimization [2]. This represents an important shift: AI is no longer merely a workload for hardware; it is becoming a participant in hardware design. Yet these systems generally search within recognizable computer-engineering spaces. RSCD proposes widening the search space beyond optimizing conventional hardware toward searching more abstract computational organizations.

2.3 Neuromorphic computation

Neuromorphic computing demonstrates why substrate-native design matters. Research on Intel's Loihi processors has shown that simply mapping ordinary feed-forward neural networks onto neuromorphic hardware often provides modest benefits, while larger gains can appear when algorithms exploit properties native to the substrate, including recurrence, spike timing, plasticity, stochasticity, and sparse event-driven activity [3]. This distinction is central to RSCD. A radically different computational substrate should not merely host an architecture designed for another machine. Instead, the intelligence should be trained from the beginning under the native constraints and opportunities of that substrate.

2.4 Optical and analog computation

Alternative physical substrates are also moving beyond narrow demonstrations. Kalinin and colleagues presented an analog optical computer combining analog electronics and three-dimensional optics for AI inference and combinatorial optimization. Their work emphasizes the benefits of co-designing computational abstractions together with the physical hardware rather than treating one as a passive implementation target for the other [4]. RSCD extends that co-design loop further to include the learning mechanism and the successor intelligence itself.

2.5 Machine-discovered learning algorithms

Even the learning rule need not remain human-designed. In 2025, Oh and colleagues demonstrated that meta-learning across populations of agents could discover a reinforcement-learning update rule that outperformed established manually designed algorithms on several benchmarks [5]. This is significant because the search space need not stop at choosing a neural architecture; it can also include how the intelligence learns. In RSCD, the learning rule is therefore treated as part of the substrate-and-agent design problem.

2.6 Open-ended computational substrates

Recent work has also begun explicitly questioning whether machine learning should remain tied to conventional feed-forward differentiable architectures. Emergent Models: Intelligence from Tiny Substrates studies simple open-ended computational systems such as cellular automata, using evolutionary search to discover useful computational behavior. The work frames these systems as an attempt to expand the design space of machine learning beyond conventional differentiable feed-forward models [6]. This closely resembles one component of the Generation Zero proposal developed below. RSCD adds a further requirement: a discovered computational substrate should become the developmental environment of a successor designer.

2.7 Evolvable and self-designing AI

Research on evolvable AI and recursive self-design increasingly treats model components, learning rules, and development processes as entities capable of evolutionary change. Muller, Steels, and Szathmary describe the possibility of AI systems whose components, learning rules, and deployment conditions themselves become subject to evolutionary processes [7]. These approaches overlap strongly with RSCD at the level of recursive

improvement. The key distinction proposed here is the explicit coupling of recursion to successive computational substrates.

3. The Proposed Gap

The literature increasingly contains AI-assisted hardware design, hardware-software co-design, unconventional computational substrates, machine-discovered learning rules, evolutionary architecture search, recursive AI improvement, open-ended computational models, and neuromorphic or analog systems designed around native computational properties.

> ***What appears less developed is their combination into the following loop: AI0 -> designs substrate S1 -> AI1 trained natively on S1 -> AI1 designs S2 -> AI2 trained natively on S2 -> repeat.***

RSCD therefore does not claim to introduce AI-designed hardware, automated learning-rule discovery, neuromorphic computing, or recursive self-improvement individually. The proposed contribution is their composition into a generational framework. The research question becomes: What happens when each computational generation becomes the developmental environment from which the next designer emerges?

4. The Recursive Substrate Co-Design Hypothesis

Let AIn denote an intelligent system capable of designing candidate computational systems, and Sn denote the computational substrate on which AIn is trained and operates. RSCD proceeds approximately as follows: AI0 operating on S0 designs candidate substrate S1. A successor intelligence, AI1, is then trained from initialization using the native computational primitives available in S1. AI1 subsequently searches for a new substrate S2. The process continues: S0 -> AI0 -> S1 -> AI1 -> S2 -> AI2 -> S3 ...

> ***The central hypothesis is that changing the substrate may alter the design space accessible to the designer.***

AI1 may therefore discover computational structures that AI0 does not efficiently discover, not merely because AI1 is more capable, but because AI1 develops under different computational constraints. This distinguishes RSCD from ordinary recursive self-improvement. The successor receives more than improved software; it receives a potentially different computational environment in which cognition develops.

5. Why Native Training Matters

Suppose an AI running on conventional GPU hardware discovers a radically different computational system. One option is to translate the existing AI onto the new machine, preserving much of the original architecture's historical baggage. The alternative is to initialize a new intelligence inside the new substrate and allow its cognition to develop around the substrate's native properties. RSCD proposes the second approach.

This distinction already appears indirectly in neuromorphic computing. Systems designed around sparse event-driven communication, local plasticity, recurrence, and temporal structure can exploit neuromorphic hardware more effectively than conventional networks mechanically translated onto the same hardware [3]. The same principle may become more important as substrates become stranger.

6. Generation Zero

The first RSCD experiment does not require fabricating new hardware. Instead, it can use a simulated substrate laboratory. A computational substrate can be represented through configurable primitives such as discrete or continuous state, synchronous or asynchronous updates, fixed or rewritable connectivity, local or global communication, persistent or decaying memory, deterministic or stochastic behavior, spatially local interactions, communication and state-change costs, oscillatory signaling, variable numerical precision, event-driven activation, local learning rules, topology growth, and connection pruning.

The automated designer is allowed to construct both the computational organization and the learning system that inhabits it from these primitives. Familiar concepts such as Transformers, RAM, processors, neural layers, or matrix multiplication should not be mandatory. They may emerge. They should not be assumed.

7. Generation Zero Tasks

Candidate systems should be tested on multiple compact cognitive tasks rather than a single large benchmark. Suitable tasks include pattern learning, associative memory, continual learning, planning, adaptation to changed rules, communication with another independently trained agent, and transfer to unfamiliar tasks. The purpose is not to produce a powerful language model. The purpose is to test whether alternative computational organizations support general learning behavior.

8. Multi-Objective Fitness

There is no universally optimal computing system. Optimization only makes sense relative to goals. Candidate substrates should therefore be evaluated according to several objectives: task capability, learning speed, energy proxy, operation count, memory requirements, communication cost, robustness, adaptability, transfer performance, and fault tolerance.

Rather than collapsing all of these into a single score, the experiment could maintain a Pareto frontier of candidate systems. Different computational families may occupy different regions of that frontier. The interesting discovery may therefore not be a single architecture, but a previously unexplored class of computation.

9. The Control Experiment

Generation Zero requires a strong control condition. One design process should receive familiar computational primitives such as matrix operations, conventional digital memory, standard neural components, and familiar optimization methods. A second process receives the broader substrate search space. Both should receive comparable computational search budgets.

If conventional computation wins, the result suggests that current architectures occupy a particularly strong region of the tested design space. If unconventional systems win only on narrow dimensions, they may reveal useful specialized computational families. If unconventional systems generalize across several cognitive tasks, the case for deeper investigation becomes substantially stronger. All three outcomes are scientifically useful.

10. The First Successor Experiment

The defining RSCD experiment begins after Generation Zero produces a promising substrate S1. Instead of porting AI0 onto S1, researchers initialize AI1 from scratch. AI1 learns using only mechanisms naturally available within S1. After reaching sufficient capability, AI1 receives a design challenge comparable to the one originally given to AI0: design a computational substrate and learning system that improves on its own generation under specified physical and computational constraints.

Researchers then compare the resulting S2 search against the earlier AI0 search. The critical question is whether AI1 systematically discovers designs that AI0 failed to find. If so, the result would provide preliminary evidence that computational substrate changes the accessible search landscape of the designer itself.

11. What Would Count as a Strong Result?

The weakest positive result would be quantitative: a discovered system performs a task more efficiently. A stronger result would be architectural. For example, the search might repeatedly discover combinations involving computation embedded directly in memory, persistent adaptive connectivity, useful stochasticity, oscillatory representations, temporary task-specific topology, dynamic growth and pruning, computation through physical-style relaxation, sparse event-driven signaling, or communication through distributed states rather than explicit symbols.

Many of these ideas already exist individually. The interesting result would be an unexpected configuration that was not explicitly designed by researchers, repeatedly emerges under optimization, survives comparison across multiple tasks, and remains useful when transferred beyond the environment in which it evolved.

12. Interpretability

A machine-designed computational system may become difficult for humans to understand. RSCD should therefore separate discovery from explanation. The primary search should not necessarily be restricted to architectures humans find intuitive, because that restriction could eliminate the systems being sought.

Instead, each promising discovery should trigger a parallel interpretability process: identify functional modules, perform ablations, trace information flow, isolate causal structures, simplify discovered mechanisms, and construct human-readable approximations. The research process therefore begins to resemble experimental science. Researchers may no longer exclusively design the mechanism; they may need to study the mechanism that was discovered.

13. Safety and Experimental Containment

RSCD does not require unrestricted autonomous self-improvement. Early experiments can remain simulated, resource-limited, isolated from external systems, incapable of self-deployment or obtaining additional compute, and incapable of modifying the host environment. The recursive loop can remain explicitly human-mediated: generate -> evaluate -> inspect -> select -> instantiate successor.

Research on evolvable AI suggests that human-controlled reproduction and selection differ substantially from uncontrolled evolutionary ecosystems, making this distinction important both experimentally and from a safety perspective [7].

14. Limitations and Open Questions

RSCD is presently a conceptual framework, not an experimentally validated method. Several limitations should be explicit. First, the available search space must still be specified by humans, so an apparently open-ended substrate search may remain strongly biased by its chosen primitives. Second, simulating alternative physical substrates on conventional hardware may distort their true efficiency and dynamics. Third, comparing distinct substrates under genuinely equal resource budgets is difficult because operation counts, memory costs, latency, energy, and precision are not directly interchangeable.

A further challenge is determining when a successor is genuinely 'native' to a substrate rather than merely another model mapped onto it. The framework also does not guarantee monotonic improvement. Successive generations may stagnate, cycle, specialize, or lose useful capabilities. Finally, the novelty claim made here is deliberately narrow: the proposal concerns the specific generational composition of substrate discovery and native successor training. A comprehensive prior-art review across artificial life, evolvable hardware, cybernetics, unconventional computing, meta-learning, and automated architecture design would still be required before making a stronger priority claim.

15. Why This Matters

> ***Most contemporary AI research asks how to build better intelligence using computers. RSCD asks a different question: What kind of computational environment would intelligence design for intelligence?***

The answer may still resemble modern computers. Digital computation has enormous practical strengths, including reliability, composability, precision, and universality. But the search may also reveal architectures that combine principles currently treated as separate research fields: computation and memory, hardware and software, learning and architecture, communication and internal state, growth and programming.

At the limit, the distinction between hardware and software may itself become less useful. Instead of a fixed computer executing a separately defined program, future systems might be better described as physical or simulated matter organized so that learning and computation occur through its dynamics.

16. Core Research Questions

1. When automated designers are allowed to jointly search computational primitives, topology, memory organization, communication, learning rules, and agent architecture, do they discover useful computational structures outside conventional neural architectures?

2. Do those structures remain competitive when tested across multiple tasks rather than the task on which they were discovered?

3. Does native training on a discovered substrate produce different learning behavior from porting an architecture developed elsewhere?

4. Does a successor designer trained natively on substrate S1 discover substrate designs that its predecessor on S0 systematically failed to discover?

5. Does repeating this process expand the reachable computational design space over successive generations?

17. Proposed Contribution

The contribution of RSCD is not the invention of automated hardware design, unconventional computing, evolutionary architecture search, machine-discovered learning rules, or recursive AI improvement. Those fields already exist. The proposed contribution is a framework that connects them across generations.

Existing research increasingly allows machines to participate in designing algorithms, architectures, hardware, learning rules, and even parts of the AI-development process itself. RSCD proposes making the resulting computational system the developmental substrate of the next designer.

> ***Design the machine. Train the successor inside the machine. Let the successor design the next machine. Repeat.***

The central scientific question is not whether each generation becomes universally more intelligent. It is whether each generation gains access to different regions of the computational possibility space. If that occurs, recursive substrate co-design could become a method for exploring forms of computation that neither human engineers nor machine designers constrained to present-day architectures would readily discover.

References

[1] Yang, J., et al. (2026). Frontis-MA1: Training an AI4AI Model towards Recursive Self-Improvement in Machine Learning Engineering. arXiv preprint arXiv:2607.28568. Source

[2] Cui, A., et al. (2026). CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research. arXiv preprint arXiv:2606.27350. Source

[3] Davies, M., Wild, A., Orchard, G., Sandamirskaya, Y., Guerra, G. A. F., Joshi, P., Plank, P., & Risbud, S. R. (2021). Advancing neuromorphic computing with Loihi: A survey of results and outlook. Proceedings of the IEEE, 109, 911-934. Source

[4] Kalinin, K. P., et al. (2025). Analog optical computer for AI inference and combinatorial optimization. Nature, 645, 354-361. Source

[5] Oh, J., Farquhar, G., Kemaev, I., Calian, D. A., Hessel, M., Zintgraf, L., Singh, S., van Hasselt, H., & Silver, D. (2025). Discovering state-of-the-art reinforcement learning algorithms. Nature, 648, 312-319. Source

[6] Bocchese, G., Giacobbo, N., Guichard, E., Wiles, J. K., & Devireddy, A. (2026). Emergent Models: Intelligence from Tiny Substrates. Publication venue not listed in the retrieved record. Source

[7] Muller, V., Steels, L., & Szathmary, E. (2026). Evolvable AI: Threats of a new major transition in evolution. Proceedings of the National Academy of Sciences of the United States of America, 123. Source